



UNIVERSITY OF MINNESOTA

Driven to Discover®

Rigid Body Dynamics

CSCI 5980: Game Engine Architecture

Evan Suma Rosenberg | CSCI 5980 | Spring 2026

This course content is offered under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International license.



Rigid Body Dynamics

- Physics engines are concerned with **dynamics**
 - Movement under the action of external forces
- Two simplifying assumptions:
 1. **Classical (Newtonian) mechanics**: Objects obey Newton's laws. No quantum or relativistic effects
 2. **Rigid bodies**: All objects are perfectly solid, non-deformable. Shape is constant
- The physics system also enforces **constraints**:
 - **Non-penetration** (objects can't pass through each other)
 - Tight interconnection between physics and collision detection

Physics World Organization

- The physics system shares the **collision world** data structure
- Typically a **one-to-one mapping** between rigid bodies and collidables
 - Havok: `hkpRigidBody` references one `hkpCollidable`
 - PhysX: `NxActor` serves as both collidable and rigid body
 - Bullet: `btRigidBody` inherits from `btCollisionObject`
- Rigid bodies are usually distinct from logical **game objects**
 - A simple object (rock, weapon) might map to one rigid body
 - An articulated character or machine might map to many interconnected rigid bodies

Units

- Most simulations use the **MKS system**
 - meters (m)
 - kilograms (kg)
 - seconds (s)
- Constants are expressed in MKS
 - For example, gravitational acceleration ($g = -9.8 \text{ m/s}^2$)
- Using other units requires making everything consistent
 - Most teams stick with MKS

Linear and Angular Dynamics

- An unconstrained rigid body has **six degrees of freedom (DOF)**
 - Translate along 3 axes
 - Rotate about 3 axes
- The motion can be separated into two independent components:
 - **Linear dynamics**: Motion ignoring all rotation (as if the body were a point mass)
 - **Angular dynamics**: Rotational motion of the body
- This is extremely helpful
 - Calculate linear motion first, then layer angular motion on top

Center of Mass

- For linear dynamics, all mass is concentrated at the **center of mass (CM)**
- The CM is the balancing point of the body for all orientations
- For uniform density, the CM lies at the **centroid**
- For non-uniform density, it is a **mass-weighted average** of positions:

$$\mathbf{r}_{CM} = \frac{\sum_{\forall i} m_i \mathbf{r}_i}{\sum_{\forall i} m_i} = \frac{\sum_{\forall i} m_i \mathbf{r}_i}{m}$$

- The CM always lies inside a convex body, but may lie **outside** a concave body

Linear Dynamics

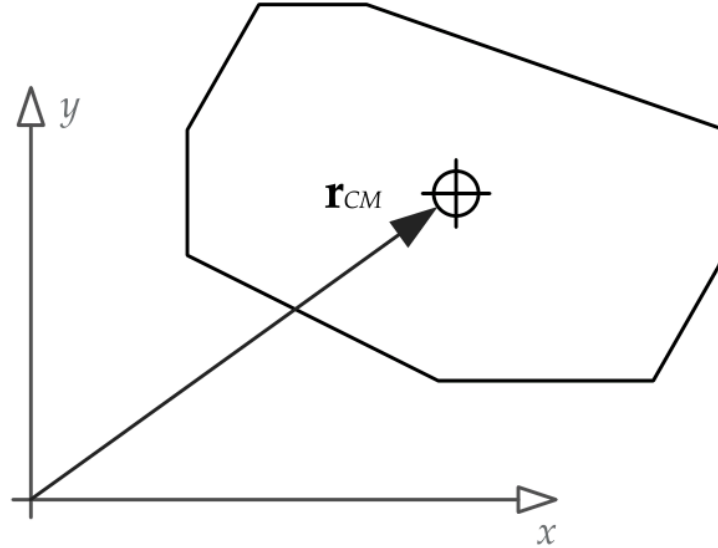


Figure 13.22. For the purposes of linear dynamics, the position of a rigid body can be fully described by the position of its center of mass.

Linear Velocity and Acceleration

- Position of a rigid body: a position vector $\mathbf{r}$ from world origin to the center of mass
- **Linear velocity** (m/s): first time derivative of position

$$\mathbf{v}(t) = \frac{d\mathbf{r}(t)}{dt} = \dot{\mathbf{r}}(t)$$

- **Linear acceleration** (m/s²): first derivative of velocity, second derivative of position

$$\mathbf{a}(t) = \frac{d\mathbf{v}(t)}{dt} = \dot{\mathbf{v}}(t) = \ddot{\mathbf{r}}(t)$$

Forces

- A **force** is anything that causes an object with mass to accelerate or decelerate.
 - Represented as a vector **F**
 - Measured in Newtons ($\text{kg}\cdot\text{m}/\text{s}^2$)
- Net force on a body is the sum of all applied forces **$F_{\text{net}} = \text{sum of } F_i$**

Force and Momentum

- **Newton's Second Law** states that force is proportional to acceleration and mass:

$$\mathbf{F}_{\text{net}}(t) = m\mathbf{a}(t) = m\mathbf{r}''(t)$$

- **Linear momentum** is the multiplication of a body's linear velocity by its mass:

$$\mathbf{p}(t) = m\mathbf{v}(t)$$

- More general form of Newton's law (for non-constant mass):

$$\mathbf{F}(t) = \frac{d\mathbf{p}(t)}{dt} = \frac{d(m(t)\mathbf{v}(t))}{dt}$$

Solving the Equations of Motion

- Central problem: find $\mathbf{v}(t)$ and $\mathbf{r}(t)$ given the net force $\mathbf{F}_{\text{net}}(t)$
- This amounts to solving a pair of **ordinary differential equations (ODEs)**
- Force can be a function of time, position, velocity, and more:

$$\mathbf{F}(t, \mathbf{r}(t), \mathbf{v}(t), \dots) = m\mathbf{a}(t)$$

- Examples:
 - **Spring force** (proportional to displacement)
 - **Damping force** (proportional to velocity)

Analytical Solutions

- In rare cases, the ODE can be solved analytically in closed form
- Example: Vertical motion under constant gravity, $\mathbf{a(t) = g = -9.8 \text{ m/s}^2}$

$$\ddot{y}(t) = g$$

$$\dot{y}(t) = gt + v_0$$

$$y(t) = \frac{1}{2}gt^2 + v_0t + y_0$$

- Analytical solutions are **almost never possible** in game physics because:
 - Closed-form solutions are not always known
 - Games are interactive, so forces are unpredictable

Numerical Integration

Numerical Integration

- Physics engines solve differential equations in a **time-stepped** manner
- Use the solution from the previous time step to find the next
- Given $\mathbf{r}(t_1)$, $\mathbf{v}(t_1)$, and $\mathbf{F}(t, \mathbf{r}, \mathbf{v})$:
 - find $\mathbf{r}(t_2)$ and $\mathbf{v}(t_2)$
 - where $t_2 = t_1 + dt$

Explicit Euler Method

- The simplest numerical ODE solution
- Multiply velocity by Δt , then add to current position:

$$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t$$

- Similarly for velocity:

$$\mathbf{v}(t_2) = \mathbf{v}(t_1) + \frac{\mathbf{F}_{net}(t)}{m} \Delta t$$

- This assumes velocity is **constant** during the time step

Properties of Numerical Methods

- **Convergence**
 - Does the approximate solution approach the real solution as Δt approaches zero?
- **Order**
 - How bad is the error? Measured by comparing with the Taylor series expansion.
 - The explicit Euler method is **first-order** (error is $O(\Delta t^2)$)
- **Stability**
 - If the method adds energy, velocities will "explode" (unstable)
 - If the method removes energy, it has an overall damping effect (stable)
- Explicit Euler has **high error and poor stability**
 - It is not used in general-purpose dynamics simulations

Alternatives to Explicit Euler

- **Backward Euler**

- Another first-order method

- **Midpoint Euler**

- A second-order method

- **Runge-Kutta** methods

- A family of higher-order methods; the fourth-order (**RK4**) is particularly popular

- The most commonly used method today is probably the **Verlet method**

Verlet Integration

- Achieves high order (low error), is relatively simple and inexpensive
- Gives position directly in terms of acceleration in **one step** (no separate velocity-to-position step)
- Derived by adding forward and backward Taylor series expansions:

$$\mathbf{r}(t_1 + \Delta t) = 2\mathbf{r}(t_1) - \mathbf{r}(t_1 - \Delta t) + \mathbf{a}(t_1)\Delta t^2 + O(\Delta t^4)$$

- Velocity is conspicuously absent, but can be approximated:

$$\mathbf{v}(t_1 + \Delta t) = \frac{\mathbf{r}(t_1 + \Delta t) - \mathbf{r}(t_1)}{\Delta t} + O(\Delta t)$$

Velocity Verlet

The more commonly used variant, a four-step process:

1. Calculate position: $\mathbf{r}(t_1 + \Delta t) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t + \frac{1}{2}\mathbf{a}(t_1)\Delta t^2$
2. Calculate half-step velocity: $\mathbf{v}(t_1 + \frac{1}{2}\Delta t) = \mathbf{v}(t_1) + \frac{1}{2}\mathbf{a}(t_1)\Delta t$
3. Determine new acceleration: $\mathbf{a}(t_2) = \frac{1}{m}\mathbf{F}(t_2, \mathbf{r}(t_2), \mathbf{v}(t_2))$
4. Calculate full-step velocity: $\mathbf{v}(t_1 + \Delta t) = \mathbf{v}(t_1 + \frac{1}{2}\Delta t) + \frac{1}{2}\mathbf{a}(t_1 + \Delta t)\Delta t$

Step 3 uses $\mathbf{r}(t_2)$ from step 1. If force is velocity-dependent, the next frame's velocity must be **approximated**.

Angular Dynamics

Angular Dynamics in 2D

- In 2D, every rigid body is a thin sheet (plane lamina).
 - All linear motion in the xy-plane
 - Single rotation angle θ (radians) about the z-axis
- For each linear quantity, there is an angular analog:

<i>Linear</i>	<i>Angular</i>
Position r	Orientation θ
Velocity v	Angular speed ω
Acceleration a	Angular acceleration α

Angular Dynamics in 2D

- Angular speed

$$\omega(t) = \frac{d\theta(t)}{dt}$$

- Angular acceleration

$$\alpha(t) = \frac{d\omega(t)}{dt}$$

Moment of Inertia

- The rotational equivalent of mass
- Measures how easy or difficult it is to change a body's angular speed about a particular axis
- If mass is concentrated **near** the axis, the moment of inertia is **smaller** (easier to rotate)
- In 2D, the moment of inertia is a simple scalar **I**

Torque

- A force applied off-center introduces a **rotational force** called **torque**, in addition to linear motion
- Torque is calculated via a cross product: $\mathbf{N} = \mathbf{r} \times \mathbf{F}$
 - $\mathbf{r}$ is the vector from the center of mass (CM) to the force's point of application
- Torque increases as the force is applied **farther from the CM** (this is why levers work)
 - A force through the CM produces **no torque** ($r = 0$)

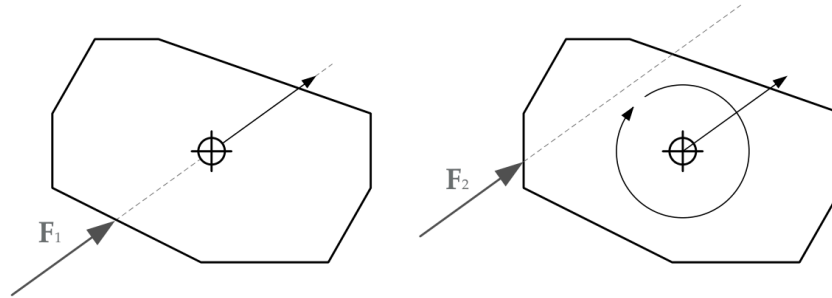


Figure 13.24. On the left, a force applied to a body's CM produces purely linear motion. On the right, a force applied off-center will give rise to a torque, producing rotational motion as well as linear motion.

Solving 2D Angular Equations

- For 2D, we can solve angular equations using exactly the same numerical integration techniques we applied to linear dynamics
- Explicit Euler solutions:

Angular:	Linear:
$\omega(t_2) = \omega(t_1) + I^{-1}N_{\text{net}}(t_1)\Delta t$	$\mathbf{v}(t_2) = \mathbf{v}(t_1) + m^{-1}\mathbf{F}_{\text{net}}(t_1)\Delta t$
$\theta(t_2) = \theta(t_1) + \omega(t_1)\Delta t$	$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t.$

- More accurate methods (e.g., velocity Verlet) can also be applied

Angular Dynamics in 3D: The Inertia Tensor

- In 3D, a body may have different mass distributions about different axes
- The rotational mass is represented by a **3x3 matrix** called the **inertia tensor** I
- Diagonal elements ($I_x x, I_y y, I_z z$) are moments of inertia about the principal axes
- Off-diagonal elements are **products of inertia**
 - Products of inertia produce physically realistic but unintuitive motions
 - The average game player would probably think were “wrong” anyway
- Thus, the tensor is often simplified to three dimensions: $I_x x, I_y y, I_z z$

Angular Velocity in 3D is Not Conserved

- In 3D, the angular velocity vector $\omega(t)$ may not be constant even **in the absence of all forces**
- The axis of rotation can continually change direction
- Try spinning a rectangular block in mid-air:
 - Spinning about the **shortest** or **longest** axis: stable, axis stays constant
 - Spinning about the **medium** axis: wildly unstable rotation

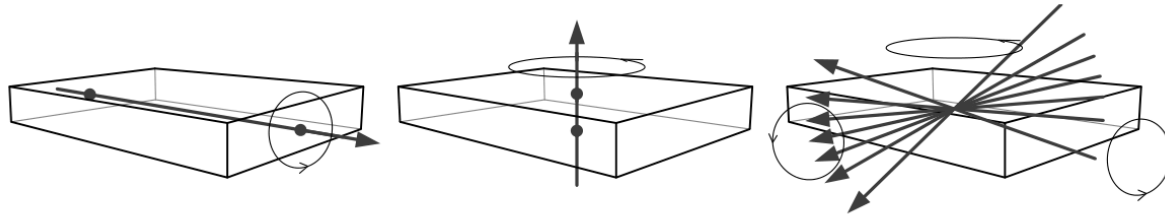


Figure 13.26. A rectangular object that is spun about its shortest or longest axis has a constant angular velocity vector. However, when spun about its medium-sized axis, the direction of the angular velocity vector changes wildly.

Angular Velocity in 3D is Not Conserved

- Angular velocity is **not conserved**, but **angular momentum $\mathbf{L}$** is:

$$\mathbf{L}(t) = \mathbf{I}\boldsymbol{\omega}(t)$$

- We solve for angular momentum $\mathbf{L}$, then derive $\boldsymbol{\omega}$ from $\mathbf{L}$ and $\mathbf{I}$
- This is a key difference from 2D, where we could solve for $\boldsymbol{\omega}$ directly

3D Angular Equations of Motion

- Orientation is a quaternion, angular velocity is a vector.
- We construct an **angular velocity quaternion** with the w component set to 0:

$$\boldsymbol{\omega} = [\omega_x \ \omega_y \ \omega_z \ 0]$$

- The differential equation relating the orientation quaternion to the angular velocity quaternion is:

$$\dot{\mathbf{q}}(t) = \frac{1}{2}\boldsymbol{\omega}(t)\mathbf{q}(t)$$

- Equations for the explicit Euler method in three dimensions can be found in your textbook.

Collision Response

Energy

- When a force moves a body over a distance, it does **work**
 - Either adds energy or removes energy from a system of rigid bodies
- **Potential energy V**
 - Energy due to position in a force field (e.g., gravitational)
- **Kinetic energy T**
 - Energy due to motion relative to other bodies
- Total energy $E = V + T$ is **conserved** in an isolated system

Impulsive Collision Response

- Most simulations use **Newton's law of restitution** for instantaneous frictionless collisions
- Simplifying assumptions:
 - Collision force acts over an **infinitesimally short** time (an idealized **impulse**)
 - **No friction** at the contact point (impulse is normal to both surfaces)
 - Complex submolecular interactions approximated by the **coefficient of restitution** ε
 - $\varepsilon = 1$: perfectly elastic, no energy lost (billiard balls)
 - $\varepsilon = 0$: perfectly inelastic/plastic, bodies stick together (putty)

Collision Response: Conservation Laws

- **Linear momentum** is conserved:

$$m_1 \mathbf{v}_1 + m_2 \mathbf{v}_2 = m_1 \mathbf{v}'_1 + m_2 \mathbf{v}'_2$$

- **Kinetic energy** is conserved (accounting for energy loss):

$$\frac{1}{2}m_1 v_1^2 + \frac{1}{2}m_2 v_2^2 = \frac{1}{2}m_1 v_1'^2 + \frac{1}{2}m_2 v_2'^2 + T_{lost}$$

- The **coefficient of restitution** relates pre- and post-collision velocities:

$$(\mathbf{v}'_2 - \mathbf{v}'_1) = \varepsilon(\mathbf{v}_2 - \mathbf{v}_1)$$

Applying the Impulse

- An impulse $\hat{\mathbf{p}}$ is applied normal to both surfaces at the contact point.
 - Body 1 receives $+\hat{\mathbf{p}}$
 - Body 2 receives $-\hat{\mathbf{p}}$

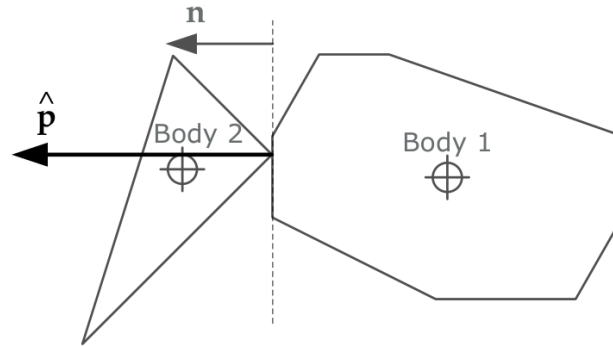


Figure 13.27. In a frictionless collision, the impulse acts along a line normal to both surfaces at the point of contact. This line is defined by the unit normal vector $\mathbf{n}$.

Applying the Impulse

- Post-collision velocities (ignoring rotation):

$$\mathbf{v}'_1 = \mathbf{v}_1 + \frac{\hat{p}}{m_1} \mathbf{n}, \quad \mathbf{v}'_2 = \mathbf{v}_2 - \frac{\hat{p}}{m_2} \mathbf{n}$$

- Solving for the impulse magnitude:

$$\hat{p} = \frac{(\varepsilon + 1)(\mathbf{v}_2 \cdot \mathbf{n} - \mathbf{v}_1 \cdot \mathbf{n})}{\frac{1}{m_1} + \frac{1}{m_2}}$$

- For a perfectly elastic collision against an infinite-mass surface, this reduces to a **reflection** of velocity about the contact normal.
- Note that this only accounts for linear velocity. The math gets more complicated when we take rotations of bodies into account!

Penalty Forces

- Imaginary **penalty forces** are an alternative approach to collision response
- Acts like a stiff damped spring between contact points of interpenetrating bodies
 - Spring constant k controls interpenetration duration
 - Damping coefficient b acts like the restitution coefficient
- Pros:
 - Easy to implement and understand
 - Works well when **three or more bodies** interpenetrate simultaneously
- Cons:
 - Forces respond only to position, so they may not align with expected direction for high-speed collisions
 - Works well for low-speed impacts, poorly for fast-moving objects

Friction

- A force resisting relative movement between two bodies in continuous contact
- Types of friction:
 - **Static**: Resistance to starting motion
 - **Dynamic/Sliding**: Resistance during sliding movement
 - **Rolling**: Acts at the contact between a round object and a surface
 - **Collision**: Acts instantaneously at the point of contact during a collision

Coming to Rest and Sleeping

- Moving objects should eventually come to rest as energy is removed via friction and damping
- In practice, floating-point error and numerical instability can cause objects to **jitter forever**
- Physics engines use heuristics to detect oscillation and remove extra energy or stop objects
- When an object reaches equilibrium, it can be **put to sleep**
 - Excluded from the simulation temporarily but still active for collision
 - Woken if a force/impulse acts on it or a supporting contact is lost

Simulation Islands

- Havok, PhysX, and Bullet group interacting (or potentially interacting) objects into **simulation islands**
- Each island is simulated independently, enabling **cache coherency** and **parallel processing**
- Entire islands are put to sleep (not individual bodies)
 - Pro: Bigger performance boost from sleeping a whole group
 - Con: If even one object in an island is awake, the whole island is awake

Constraints

Constraints

- An unconstrained rigid body has **6 DOF**.
 - Constraints reduce degrees of freedom.
- Examples of constraint-driven behaviors:
 - A swinging chandelier (point-to-point)
 - A door that can be kicked, slammed, blown off its hinges (hinge)
 - A vehicle's wheel assembly (axle with damped springs)
 - A train pulling a trailer (stiff spring/rod)
 - A rope or chain (chain of stiff springs/rods)
 - A rag doll (specialized skeleton joint constraints)

Common Constraint Types

- **Point-to-point**: Ball-and-socket joint. A point on body A must align with a point on body B
- **Stiff spring**: Like point-to-point but keeps the points separated by a specified distance (an invisible rod)
- **Hinge**: Limits rotation to a single DOF about the hinge axis
- **Prismatic**: Piston-like; restricts motion to a single translational DOF.
- **Planar**: Objects constrained to move in a 2D plane
- **Wheel**: Hinge with unlimited rotation plus vertical suspension via spring-damper
- **Pulley**: Imaginary rope through a pulley attached to two bodies, moving via a leverage ratio
- Constraints may be **breakable**: they come apart after enough force, or the game can toggle them on/off

Integrating a Physics Engine

Linking Game Objects and Rigid Bodies

- Rigid bodies are abstract mathematical descriptions.
 - They must be linked to visual representations via **game objects**
- The linkage is indirect:
 - Rigid Body <-> Game Object <-> Mesh Instance <-> Rendering Engine
- A game object may have:
 - **Zero rigid bodies**: No collision representation (decorative objects, unreachable geometry).
 - **One rigid body**: Simple objects. Shape approximates visual form.
 - **Multiple rigid bodies**: Complex objects with pieces that move relative to each other.

Physics-Driven Bodies

- Objects whose motion is driven **entirely by the simulation**
 - Debris, exploding buildings, rolling rocks, shell casings
- After stepping the simulation, query the physics system for each body's position and orientation, then apply it to the game object or skeleton joint
- Example: A safe with a detachable door
 - Two rigid bodies (housing + door), linked by a hinge constraint
 - Physics drives both joint transforms via the skeleton
 - To blow off the door: break the constraint, apply impulses

Game-Driven Bodies

- Objects moved by game code that should participate in collision but not be affected by physics
- Treated as **infinitely massive** (mass = 0 in the SDK)
 - Forces/impulses never change their velocity
- Havok calls these **key framed** bodies
 - Cannot simply set position each frame (introduces discontinuities).
 - Instead, move them using **impulses** that position them correctly at end of the time step.
 - Must zero out velocity when the body should stop.
- Bullet calls them **kinematic objects**
 - Allows you to update the transform directly
 - Automatically calculates velocity from transform deltas

Motion Type

- In Havok, every `hkpRigidBody` has a **motion type** field:
 - **Fixed**: Collision only, never changes
 - **Key framed** (game-driven): Infinite mass, moved via impulses
 - **Dynamic** (physics-driven): Participates fully in the simulation
- In Bullet, the motion type is set using **collision flags**:
 - `btRigidBody` inherits this from `btCollisionObject`
 - `CF_DYNAMIC_OBJECT` , `CF_STATIC_OBJECT` , or `CF_KINEMATIC_OBJECT`

Updating the Simulation

Each frame, the following steps are performed (typically in this order):

1. **Update game-driven rigid bodies:** Match transforms to game objects/joints
2. **Update phantoms:** Move phantom shapes to current positions
3. **Update forces, apply impulses, adjust constraints:** Apply game-driven forces, break constraints as needed
4. **Step the simulation:** Integrate, detect collisions, resolve collisions, solve constraints
5. **Update physics-driven game objects:** Extract rigid body transforms, update game objects
6. **Query phantoms:** Read contacts, make decisions
7. **Perform collision casts:** Run ray/shape casts for various systems

Timing and Frequency

- Most numerical integrators and constraint solvers work best with a **constant dt**
- Best practice: step with an ideal dt (1/30 or 1/60 second) and govern the game loop's frame rate
- If frame rate drops below target, let the physics **slow down visually** rather than adjusting the time step
- Collision queries (ray/shape casts) ideally run **after** the physics step, but the step runs late in the frame
- Options include:
 - Base decisions on **last frame's state**
 - Accept a **one-frame lag**
 - Run queries **after the physics step** when decisions can be deferred

Example Uses in Games

Bullet Traces

- **Ray cast approach:** Fire a ray cast on the frame the weapon fires, determine the hit, apply impact immediately
 - Does not account for travel time or gravity arc
- **Rigid body approach:** Model projectiles as physics objects moving through the world
 - Better for slow-moving projectiles (thrown objects, rockets)
- Issues to consider:
 - In third-person games, the ray from the gun may not align with the ray from the camera
 - Mismatches between collision and visible geometry can block shots the player thinks should hit
 - AI may need to "lead" shots for finite travel time

Grenades and Explosions

■ Grenades:

- Sometimes free-moving physics objects, but this loses control
- Can impose artificial forces (e.g., extreme drag after first bounce)
- Some teams manage grenade motion entirely manually with pre-calculated arcs

■ Explosions

- Visual effects + audio + a growing damage radius
- Apply **radial impulses** to objects within the radius (normalized direction from epicenter, scaled by magnitude, falling off with distance)
- May interact with other systems (e.g., foliage bending from the shock wave)

Destructible Objects

- Start undamaged (single cohesive object), break into separate pieces
- Typically: divide a model into breakable pieces, assign a separate rigid body to each
- May swap between "undamaged" and "damaged" visual/collision models
- Features beyond simple stacking:
 - Indestructible pieces (base, chassis), non-structural pieces (fall off when hit), structural pieces (propagate forces to pieces on top), explosive pieces
 - Health per object or per piece
 - Breakable constraints for hanging pieces
 - Time-delayed collapse propagation (e.g., a bridge collapsing from one end)

Character Mechanics

- Humanoid characters are usually **not** modeled as free-moving physics objects
- Instead: game-driven capsule-shaped rigid bodies linked to skeleton joints, primarily for bullet hit detection
- Movement via **sphere or capsule casts** probing in the direction of desired motion:
 - Slide along walls at oblique angles
 - Pop up over low curbs
 - Prevent "falling" state when walking off a low curb
 - Prevent walking up slopes that are too steep
- Havok's character controller models a character as a **capsule phantom** with a cleaned-up contact manifold
- Bullet also includes a `btKinematicCharacterController`

Camera Collision

- Surround the virtual camera with **sphere phantoms or sphere casts** to detect potential collisions
- Responses include adjusting position and/or orientation to avoid passing through objects
- Key considerations:
 - **Zooming in** works well in many situations (can go to first-person view)
 - Avoid drastic **horizontal angle** changes (messes with camera-relative controls)
 - Limit **vertical angle** changes (player loses track of the horizon)
 - Handle objects **between camera and character** (make them translucent, zoom in, swing around)
- Camera collision is incredibly tricky and requires lots of trial and error

Rag Doll Integration

- When a character becomes unconscious, the rag doll system kicks in
 - Physics simulates rigid bodies, which drive skeletal joints
- The conscious and unconscious collision models may differ entirely
 - Conscious: overlapping game-driven bodies (no holes for enemies to shoot through)
 - Unconscious: non-overlapping bodies (to avoid explosive impulses from interpenetration)
- Transitioning from animation to rag doll is tricky
 - Simple LERP blending usually fails because physics poses diverge quickly
 - **Powered constraints** during the transition can help
 - Detect collisions early to drop into rag doll mode before limbs get stuck inside geometry

Advanced Physics Features

- **Deformable bodies:** DMM and similar engines simulate non-rigid objects
- **Cloth:** Modeled as sheets of point masses connected by stiff springs
- **Hair:** Simulated filaments or cloth sheets textured as hair
- **Water and buoyancy:** Surface simulation (often rendering-only), buoyancy via forces
- **Fluid dynamics:** Specialized simulation libraries, increasingly used in games (LittleBigPlanet, PhysX)
- **Physically based audio synthesis:** Dynamically generating collision/rolling/sliding sounds
- **GPGPU:** Harnessing GPU parallel processing for collision and physics

Key Takeaways

- Rigid body dynamics separates into **linear** and **angular** components. Linear dynamics describes the motion of the center of mass; angular dynamics describes rotation.
- **Numerical integration** (explicit Euler, Verlet, RK4) solves the equations of motion in discrete time steps. Method choice affects accuracy, stability, and performance.
- Collision response uses **impulses** (instantaneous velocity changes) or **penalty forces** (spring-damper systems). The **coefficient of restitution** controls energy loss.
- **Constraints** (point-to-point, hinge, prismatic, rag doll) reduce degrees of freedom and enable complex mechanical behaviors. A constraint solver iteratively minimizes error.
- Rigid bodies connect to game objects as **physics-driven** (simulation controls motion), **game-driven** (animation/player controls motion), or **fixed** (static world geometry).
- **Sleeping** and **simulation islands** optimize performance by excluding settled objects.



Questions?